Lesson 5

Material

https://github.com/unixfile/learning-python-2026

Today

- Sets and set operations
- Dictionaries
- Exception handling
- Reading stack traces

1 Sets

```
In : my_set = {3, 7, 'gamma', 'echo'}
In : type(my_set)
Out: set
```

- A set is an unordered collection of **unique** elements
- Sets are mutable

2 Set idiosyncrasies

```
In : fruits = {'banan', 'banan', 'melon', 'kiwi'}
In : fruits
Out: {'banan', 'melon', 'kiwi'}
```

- Duplicates are silently removed
- Elements have no guaranteed order, so indexing is not possible

```
In : fruits[0]
...
TypeError: 'set' object is not subscriptable
```

3 Adding and removing

```
In : countries = {'Brasilien', 'Chile'}

In : countries.add('Bolivia')
In : countries
Out: {'Brasilien', 'Chile', 'Bolivia'}

In : countries.update({'Brasilien', 'Paraguay'})
In : countries
Out: {'Brasilien', 'Chile', 'Bolivia', 'Paraguay'}

In : countries.remove('Sverige')   # KeyError if missing
In : countries.discard('Sverige')  # silent no-op if missi
```

4 Set operations

```
In : low = {1, 2, 3, 4}
In : high = {3, 4, 5, 6}

In : low | high   # union
Out: {1, 2, 3, 4, 5, 6}

In : low & high   # intersection
Out: {3, 4}

In : low ^ high   # symmetric difference
Out: {1, 2, 5, 6}

In : low - high   # difference
Out: {1, 2}
```

5 Set comprehensions

```
In : squares = {x**2 for x in range(6)}
In : squares
Out: {0, 1, 4, 9, 16, 25}

In : evens = {x for x in range(10) if x % 2 == 0}
In : evens
Out: {0, 2, 4, 6, 8}
```

- Same syntax as list comprehensions but with {}

6 Dictionaries

```
In : fruit_colors = {
   :     'äpple':   'grön',
   :     'banan':   'gul',
   :     'vindruva': 'lila',
   : }
In : type(fruit_colors)
Out: dict
```

- A dictionary stores key-value pairs
- Dictionaries are mutable and ordered (since Python 3.7)

7 Accessing values

```
In : fruit_colors['banan']
Out: 'gul'

In : fruit_colors['citron']                    # KeyError!
In : fruit_colors.get('citron')                # None, no crash
In : fruit_colors.get('citron', 'okänd')
Out: 'okänd'
```

- `[]` raises `KeyError` when the key is missing
- `.get()` returns `None` (or a default) instead

8 Looping over dictionaries

```
for fruit in fruit_colors:
    print(fruit)
# äpple
# banan
# vindruva

for color in fruit_colors.values():
    print(color)
# grön
# gul
# lila

for fruit, color in fruit_colors.items():
    print(f"{fruit}: {color}")
# äpple: grön
# banan: gul
# vindruva: lila
```

9 Modifying dictionaries

```
In : fruit_colors['äpple'] = 'röd'    # update existing
In : fruit_colors['mango'] = 'orange' # add new key

In : fruit_colors.pop('banan')
Out: 'gul'

In : fruit_colors.update({'kiwi': 'grön', 'äpple': 'grön'})
```

10 Nested dictionaries

```
In : betyg = {
   :      'Alice': {'Matte': 85, 'Kemi': 92},
   :      'Bob':   {'Matte': 78, 'Kemi': 88},
   : }

In : betyg['Alice']['Matte']
Out: 85

In : betyg['Bob']['Engelska'] = 87
```

11 Dictionary comprehensions

```
In : squares = {x: x**2 for x in range(5)}
In : squares
Out: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

In : original = {'äpple': 1, 'banan': 2, 'körsbär': 3}
In : swapped = {v: k for k, v in original.items()}
In : swapped
Out: {1: 'äpple', 2: 'banan', 3: 'körsbär'}
```

12 Exception handling

```
numbers = [1, 2, 3]

try:
    print(numbers[5])
except IndexError:
    print("Index out of range")
```

- `try` runs the block; if an exception is raised Python jumps to `except`
- Code after the raising line inside `try` is skipped

13 Multiple exception types

```
try:
    value = int("abc")
    numbers = [1, 2, 3]
    print(numbers[5])
except ValueError:
    print("Not a valid number")
except IndexError:
    print("Index out of range")

# Catch several types in one handler
try:
    ...
except (ValueError, IndexError) as e:
    print(f"An error occurred: {e}")
```

14 Stack traces

```
Traceback (most recent call last):
  File "example.py", line 10, in <module>
    main()
  File "example.py", line 7, in main
    process_data()
  File "example.py", line 4, in process_data
    value = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

- The last line tells you what went wrong
- The trace shows the call chain from outermost to innermost
- Read from the bottom up, where the error usually is